

Analysis: Alien Rhyme

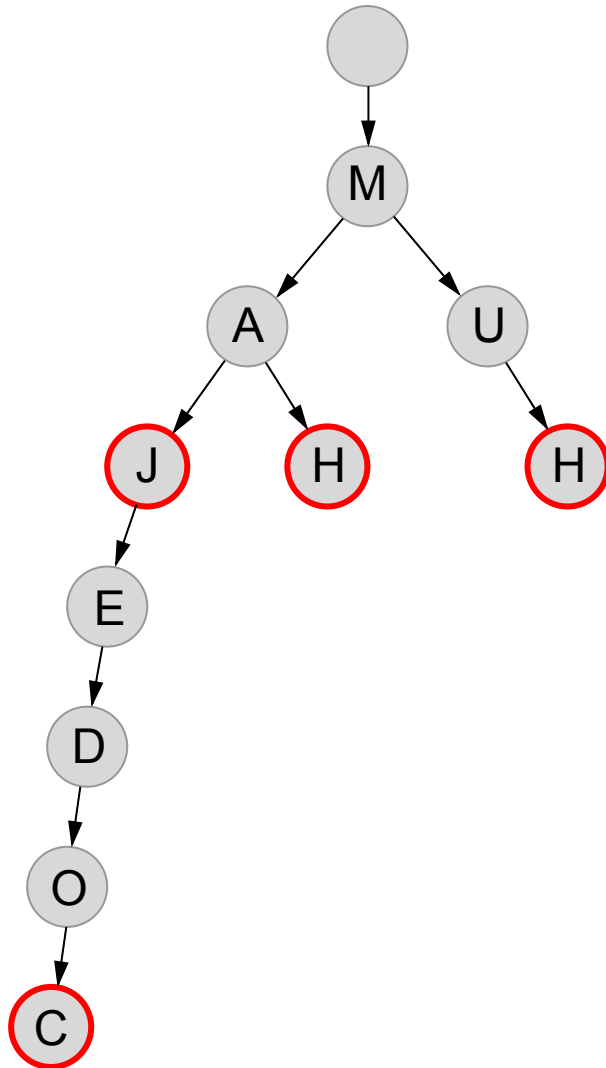
Test set 1

In the first test set there are only up to $N = 6$ words with up to 50 characters in each of them. We can simply use brute force to try all ways of grouping words into pairs (allowing some words not to be in any pair – we will effectively discard these words), try all choices of an accent-suffix for every pair, and check that none of the pairs have the same accent-suffix. Finally, we choose the maximum size across all valid groupings.

Test set 2

Let's notice how the size of an accent-suffix affects the chance of multiple words sharing it. In case we have words `CODEJAM`, `JAM`, `HUM` and `HAM`, accent-suffix `JAM` can be part of only two words, whereas a shorter accent-suffix `M` fits all four words. This leads to the following observation: for two words that we want to pair, it is never suboptimal to choose their longest available common suffix as the accent-suffix – this way we are still making sure that they rhyme, and we are allowing shorter accent-suffixes to be used by other pairs. Notice that any other pair that could use the longer suffix can also use any shorter suffix. For example, if we want words `CODEJAM` and `JAM` rhyme, we should choose `JAM` as their accent-suffix, and allow suffix `M` to be potentially used by `HUM` and `HAM`.

In this problem it is all about common suffixes of the words. In order to better operate with word suffixes, let's actually reverse the words first (so now original word suffixes are prefixes of reversed words), and build a [trie](#) (also often called prefix tree) on the reversed words. This is how a trie containing the words `CODEJAM`, `JAM`, `HUM` and `HAM` looks:



Let's also mark the trie nodes where some of the input words end. Since we are guaranteed that all the words are unique, we can use a simple boolean flag. In the picture above, trie nodes where a word ends are marked in red.

Now we can solve the problem as follows: for a trie node v , let $f(v)$ be the minimum possible number of unpaired words that use accent-suffixes whose reverses end in the node v or the subtree under it. The answer to the problem is then $N - f(\text{root})$, since $f(\text{root})$ represents all usable accent-suffixes.

How do we calculate the values of $f(v)$? If v does not have any children nodes, we set $f(v)$ to be 1, since we know that in our trie all leaf nodes are the end of a word. If node v has children, we can calculate $f(v)$ with the following algorithm assigning the result to r :

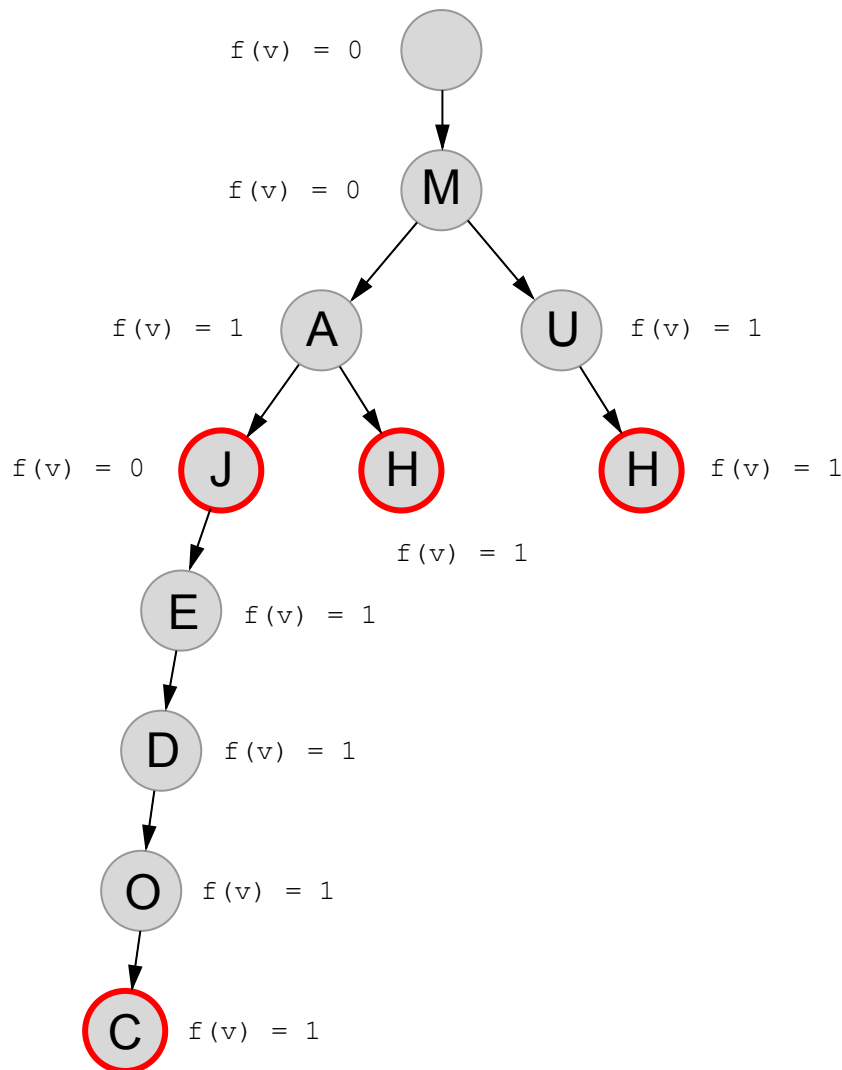
```

r = sum(f(c) for all c where c is a child node of v)
if node v is marked (there is a word that ends at v): r = r + 1
if v is not the root and f(v) ≥ 2: r = r - 2

```

First, we simply count the number of unpaired words recursively. Finally, we let two of those words be paired using the prefix that ends in the node v , which represents suffixes of original words, as the accent-suffix.

In our example trie we would get the following values of $f(v)$:



Proving the algorithm above correctly calculates $f(v)$ is straightforward. First notice that only words that are represented in the trie at or below v are pairable with the set of accent-suffixes represented by v or its subtree. Then we can proceed by induction: f is pretty clearly correct for a single node tree, as there is no pairing possible. Assume by the inductive hypotheses f works correctly on all proper subtrees under v . The pairing implied by the construction of $f(v)$ — adding any remaining pair of words to the recursive result — is valid: we are only pairing two words with the accent-suffix represented by v , and the rest is valid by the inductive hypotheses. To show that the pairing it is also of maximum size, notice that, by inductive hypothesis, there is no way to pair more than $\sum(f(c))$ for all c where c is a child node of v words with accent-suffixes that are represented by the subtree but not by v directly. This is because words in different subtrees of v cannot be matched with a longer accent-suffix than the one represented by v , and the accent-suffix represented by v can add at most a single pair to the total.

Note how we calculate the values of $f(v)$ in a recursive manner, and $f(v)$ is calculated exactly once for each possible v . Since the algorithm itself takes constant time in addition to the time of the recursion, we can calculate all $f(v)$ values in $O(T)$ time, where T is a total number of nodes in the trie. We can bound T by the total length of all words, or by $N \times m$ where m is the maximum word length.

Finally, there are less efficient but simpler implementations that also work. For example, sort the reversed words alphabetically and take any two adjacent words with a longest common prefix, pair them, remove them from the list, and repeat. This simple-to-implement algorithm basically constructs the same pairing our recursive formulation does. This shifts some implementation

complexity onto the correctness proof. If you are faster with proofs than with code, it might be an overall gain in solving speed.

Analysis: Golf Gophers

Test set 1

The difficulty of this problem stems from the fact that the windmills can loop around. If we have a windmill with 5 blades, and we find it with blade number 2 pointing downward in the morning, does that mean that 2 gophers rotated it? or that 7 did? or 12? In general, if a windmill has B blades, and we find it in position P , all that we can say for sure is that some number $(P + K \times B)$ of gophers (for some integer K) tampered with it during the night.

Because of this, we might reason that we should put as many blades as possible (that is, 18) on each windmill, and then hope that none of those windmills is visited by more than 17 gophers, in which case the total number of rotations of all blades equals the total number of gophers. This turns out to be a reasonable hope! It is hard to directly calculate the probability that none of the 18 blades will be turned more than 17 times on a given night, but a quick simulation can tell us that even in the worst case of 100 gophers, the probability of this happening is about 0.00017. Since we can run this same experiment 365 times and take the maximum result that we find, the chances of a wrong answer (due to getting a misleading result all 365 times) are infinitesimally small.

Test set 2

In test set 2, we only have 7 nights to work with, and the number of gophers can be quite large, so we cannot reasonably hope that the windmills will not loop. Now what? We might consider using different numbers of blades on each windmill during a given night, but it's hard to see how that buys us anything.

Suppose that, on one night, we try putting two blades on every windmill. At first this doesn't seem to help — shouldn't the resulting data should be almost pure noise? However, we can observe that if the number of gophers is odd, the total number of turns (across all windmills) will be odd, and if the number of gophers is even, that total number will be even. So we have a way of determining the parity of the number of gophers, no matter how they happen to turn the blades!

We can extend this idea to find out how many gophers there are modulo any number of our choice between 2 and 18. Since we only get 7 nights, though, we should choose our numbers carefully. One promising idea is to make them all prime, and there are seven primes in the range we can use: 2, 3, 5, 7, 11, 13, and 17. Then we can try to use the construction suggested by the [Chinese remainder theorem](#) to uniquely determine the number of gophers. However, this method would only work for any number of gophers up to $2 \times 3 \times \dots \times 17 = 510510$; it cannot distinguish 510511 gophers from 1 gopher! We are in trouble, since we might have up to 10^6 gophers.

The final insight that we need is that the Chinese remainder theorem only requires its moduli to all be *relatively* prime, not necessarily prime. So we can use 16 in place of 2, and 9 in place of 3. This lets us identify any number of gophers up to $5 \times 7 \times 9 \times 11 \times 13 \times 16 \times 17 = 12252240$, which easily covers what we need. (We can also get away with using the numbers 12 through 18, for example; we leave the details to the reader.)

Notice that we do not really need to do any calculations based on the Chinese remainder theorem; since the number of possible answers is relatively small, we can check all of them until we find the one that has the appropriate residue for each of our chosen moduli.

Test set 1 appendix

To prove that the probability of at least one blade rotating more than 17 times is about 0.00017, we can solve a related problem:

"Count the number of integer arrays of length L such that each array element is one of 1, 2, ..., U and each number appears at most B times in the array."

This related problem is equivalent to counting [integer partitions](#) with constraints on the size of each partition.

We can solve this related problem using [Dynamic Programming](#). First, we choose how many times U appears in the array; say it appears K times, with $0 \leq K \leq \min(B, L)$. Once we know K , we have to choose where those K values go. Using [combinations](#), we can see that there are $\binom{L}{K}$ possible options. For each option, we can treat the $L-K$ other spaces as an array that must contain the numbers 1, 2, ..., $U-1$ such that no number appears more than B times, which is a subproblem of our original problem, so we may recurse. If we sum over all possible values for K , we have the total number of valid arrays.

If we solve the above problem with $L = 100$, $U = 18$, $B = 17$, we see that there are 336647783248234011860927063629187654598455062446560501834487820535956663161762533555609870639313859125191714928476256 (or approximately 3.366×10^{125}) arrays such that no number appears more than 17 times. These arrays can represent which windmill the gophers select in a *good* configuration out of the 18^{100} possible configurations. This gives us the probability quoted above.

Analysis: Pylons

Test set 1

There are a few impossible cases for this problem. If we experiment with some small grids, we can find that in addition to the 2 x 2 grid given as a sample case, the 2 x 3, 2 x 4, and 3 x 3 cases have no solution. (Because of symmetry, the 3 x 2 and 4 x 2 cases are also impossible.)

In the 2 x 3 grid, we can notice that the middle cell of the top row shares a row, column, or diagonal with every other cell; the same is true of the central cell of the 3 x 3 grid. In each case, we cannot go from that cell to any other cells, or vice versa, so the grid is unsolvable.

In the 2 x 4 grid, we can try starting in the second cell of the top row; then we find that we are forced into a series of three moves that eventually take us to the third cell of the bottom row, from which there are no other moves; we can never move out of that set of four cells that we visited. Since the same is true of the other set of four cells, there is no solution.

The other cases in test set 1 are all solvable, though. One strategy is to make mostly "knight moves" — two unit cells in one direction, and one unit cell in the other. We may not be able to solve a grid using only knight moves — see [this article](#) for more information — so we can sprinkle in other legal moves as well. For instance, we can solve the 3 x 4 case by visiting the cells in the following order:

```
02 05 10 07
09 12 01 04
06 03 08 11
```

And here is a solution for the 3 x 5 case, which makes many steps that are wider than knight moves:

```
04 14 09 12 07
06 11 01 15 03
02 08 13 10 05
```

Because of symmetry, the only remaining cases are 2 x 5, 4 x 4, and 4 x 5, and we can solve these by hand if we choose, or we can use a brute force algorithm.

Test set 2

There are multiple strategies for dealing with test set 2. One class of approach is *constructive*. For example, we can devise general solutions for 2 x N and 3 x N grids, for arbitrary N, and then divide up the grid into horizontal strips of height 2, plus one more strip of height 3 if needed. Unfortunately, this can be tricky to get right. We need to make sure that the solutions to the subproblems do not violate the rules — the last move in one subproblem cannot be in the same row, column, or diagonal as the first move in another. Moreover, we might struggle to come up with our general 2 x N and 3 x N solutions.

The Code Jam team's first successful constructive solution included multiple cases for 2 x N (depending on whether N is odd, 0 mod 4, or 2 mod 4), and multiple cases for 3 x N (which entailed adding pairs of columns to the left and right of our hardcoded 3 x 4 and 3 x 5 solutions, bouncing between those columns to avoid breaking rules). It also made each such solution start near the left edge of each strip and end near the right edge, to avoid diagonal interactions among subproblems / strips.

Is there an easier way? Let's take a step back. The problem imposes some constraints, but one can observe that it is not too difficult to solve the larger test set 1 cases by hand. This suggests that there are many possible solutions, and we might expect even more as the grid's dimensions get larger. (We could use [Ore's theorem](#) to establish the existence of at least one solution for sufficiently large grids.) So, our intuition might suggest at this point that the best approach is some kind of brute force.

We might consider using backtracking solutions. One possible concern is that these solutions, whether they are breadth-first or depth-first, proceed in an orderly fashion that might be more likely to leave us with tough "endgames" in which the constraints are impossible. For example, if our solution somehow solves all but the bottom row of the grid, then all hope for the universe is lost!

We can try these solutions anyway, or we can rely on our occasional friend, randomness! We can pick a random starting cell, repeatedly choose valid moves uniformly at random from the space of all allowed moves from our current cell, and, if we run out of available moves, give up and start over. For any case except for the impossible ones mentioned above, this approach finds a solution very quickly.

Many problems cannot be approached with random or brute-force solutions, but identifying the problems that can be (in Code Jam or the real world) is a useful skill!

There's even a greedy algorithm

We are aware of other approaches. For example, we know that there is at least one implementation of the following idea that solves every possible test in the test set 1 and 2 limits: repeatedly greedily select an unvisited cell that has the largest count of unvisited "neighbors", where we define a cell's neighbors as those cells that share a row, column, or diagonal with that cell. (We can either rule out specific impossible cases beforehand, since there are not very many, or infer impossibility by comparing the number of unvisited cells with the largest count of unvisited neighbors.)

Although we do not provide a proof here, intuitively, it is advantageous to prevent any one row, column, or diagonal from having too many unvisited cells, relative to other rows, columns, and diagonals. For example, if we are close to the end of our journey, and a majority of our remaining unvisited cells are in the same column, we are doomed.

Given the relatively small number of possible test cases for this problem, it is not too hard to check all of them to verify that a solution works, and that may be much easier than proving correctness! That is a rare luxury to have for a Code Jam problem!